3693. Climbing Stairs II

Daily Question - November 10, 2026

Problem Statement

You are climbing a staircase with n + 1 steps, numbered from 0 to n. You are also given a 1-indexed integer array costs of length n, where costs[i] is the cost of step i.

From step i, you can jump only to step i + 1, i + 2, or i + 3. The cost of jumping from step i to step j is costs[j] + (j - i)^2.

You start from step 0 with cost = 0. Return the minimum total cost to reach step n.

Examples

Example 1: n = 4, costs = [1,2,3,4]. Output: 13. One optimal path is 0 -> 1 -> 2 -> 4, with costs 2 + 3 + 8 = 13.

Example 2: n = 4, costs = [5,1,6,2]. Output: 11. One optimal path is 0 -> 2 -> 4, with costs 5 + 6 = 11.

Example 3: n = 3, costs = [9,8,3]. Output: 12. The optimal path is 0 -> 3, with cost 3 + 9 = 12.

Constraints

1 <= n == costs.length <= 100000; 1 <= costs[i] <= 10000.

Solution: Dynamic Programming

Let dp[i] be the minimum cost to reach step i. Since a jump can advance by 1, 2, or 3 steps, dp[i] depends only on dp[i - 1], dp[i - 2], and dp[i - 3].

Initialize dp[0] = 0. For every step i from 1 to n, try each predecessor j in {i - 3, i - 2, i - 1} that is nonnegative:

dp[i] = min(dp[j] + costs[i - 1] + (i - j)^2)

The answer is dp[n]. Because costs is 1-indexed, arriving at step i uses costs[i - 1].

Complete Java Solution

```
import java.util.Arrays;

class Solution {
    public int climbStairs(int n, int[] costs) {
        long[] dp = new long[n + 1];
        Arrays.fill(dp, Long.MAX_VALUE);
        dp[0] = 0;

        for (int step = 1; step <= n; step++) {
            for (int previous = Math.max(0, step - 3);
                    previous < step;
                    previous++) {
                long jumpDistance = step - previous;
```

```
                long jumpCost = costs[step - 1]
                        + jumpDistance * jumpDistance;

                dp[step] = Math.min(
                        dp[step],
                        dp[previous] + jumpCost
                );
            }
        }

        return (int) dp[n];
    }
}
```

Correctness

Every valid route to step i must arrive from one of steps i - 1, i - 2, or i - 3. Conversely, each such predecessor can reach step i with exactly one allowed jump. Therefore, taking the minimum over these three predecessors and adding the corresponding jump cost gives the minimum cost to reach step i.

The base case dp[0] = 0 is correct because the journey starts there with no accumulated cost. By induction, every dp[i] is the minimum possible cost to reach step i, so dp[n] is the answer.

Complexity

Time: O(n), because each step examines at most three previous states. Auxiliary space: O(n) for the DP array. It can be reduced to O(1) by retaining only dp[i - 3], dp[i - 2], and dp[i - 1].

Implementation Notes

Use long for dp values and jump-cost calculations to avoid intermediate overflow. With n up to 100000 and costs[i] up to 10000, the total cost can exceed the int range. The final answer is guaranteed to fit in an int, so cast dp[n] at the end.